

TUTORIAL 9: Implementing a Generative Model for Creating Personalized Marketing Content

Varun Gadi

RA2211027010203

Section AD2

Department of Data Science and
Business Systems

1. Introduction

This tutorial demonstrates the application of generative AI techniques in creating personalized marketing content. A sequence-to-sequence model based on **LSTM (Long Short-Term Memory)** networks is trained to generate marketing text. This generative model is capable of learning from existing email content and then generating new, contextually relevant emails or advertisements.

Objectives:

- Build and train a generative AI model for marketing content generation using the Enron Email dataset.
- Visualize training loss to ensure convergence.
- Evaluate the model's text generation capabilities based on input seed text.

2. Dataset Description & Preprocessing

2.1 Dataset Overview

The model is trained on the **Enron Email Dataset**, which contains email communication data. The dataset is used to simulate personalized marketing content generation. This dataset provides diverse examples of email content across various topics and contexts.

```
[ ] import pandas as pd

# Load the dataset into a DataFrame
df = pd.read_csv('/content/enron_data/emails.csv')

# Preview the dataset to check its structure
print(df.head())
```

	file	message
0	allen-p/_sent_mail/1.	Message-ID: <18782981.1075855378110.JavaMail.e...
1	allen-p/_sent_mail/10.	Message-ID: <15464986.1075855378456.JavaMail.e...
2	allen-p/_sent_mail/100.	Message-ID: <24216240.1075855687451.JavaMail.e...
3	allen-p/_sent_mail/1000.	Message-ID: <13505866.1075863688222.JavaMail.e...
4	allen-p/_sent_mail/1001.	Message-ID: <30922949.1075863688243.JavaMail.e...

2.2 Data Preprocessing

Preprocessing steps performed:

- **Cleaning text:** Non-alphabetic characters removed, and all text was converted to lowercase.

- **Tokenization:** Text was split into characters for sequence generation.
- **Character Encoding:** A dictionary was created to map characters to integers, necessary for input into the LSTM model.

```
# Assuming the email content is in a column named 'message' (adjust if necessary)
df['cleaned_message'] = df['message'].apply(clean_text)

# Join all cleaned messages into a single text corpus
corpus = ' '.join(df['cleaned_message'].values)

# Display a portion of the cleaned corpus
print("Sample of cleaned corpus:", corpus[:500]) # Display a portion of the cleaned text
```

Sample of cleaned corpus: messageid javamailevansthyme
date mon may pdt
from phillipallenroncom
to timbeldenroncom
subject
mimeversion
contenttype textplain charsetusascii
contenttransferencoding bit
xfrom phillip k allen
xto tim belden tim beldenronronxgate
xcc
xbcc
xfolder phillipallenjanallen phillip ksant mail
xorigin allenp
xfilename pallen nonprivilegedpst

here is our forecast

messageid javamailevansthyme
date fri may pdt
from phillipallenroncom
to johnlavoratoenroncom
subject re
mimeversi

Visualizations:

- **Sample of Cleaned Corpus:** Shows the cleaned text, focusing on key features like message IDs, timestamps, and email content.
- **Character to Integer Mapping:** A dictionary was created to map characters to integers for model input.

```
# Convert X and y to numpy arrays and reshape X
X = np.reshape(X, (len(X), sequence_length, 1)) # Reshaping for LSTM input: [samples, time steps, features]
X = X / float(len(chars)) # Normalize input data (between 0 and 1)

# One-hot encode the output labels (the next character)
y = to_categorical(y, num_classes=len(chars))

# Display the shapes of X and y
print(f"Shape of X: {X.shape}")
print(f"Shape of y: {y.shape}")
```

Sample character-to-int mapping: {'\t': 0, '\n': 1, ' ': 2, 'a': 3, 'b': 4, 'c': 5, 'd': 6, 'e': 7, 'f': 8, 'g': 9}
Shape of X: (999950, 50, 1)
Shape of y: (999950, 29)

Sample of Cleaned Corpus:

```
messageid javamailvansthyme  
date mon may pdt  
from phillipallenroncom  
to timbeldenroncom  
subject mimeversion  
contenttype textplain charsetusascii  
contenttransferencoding bit  
from phillip k allen  
...
```

3. Exploratory Data Analysis (EDA)

EDA was performed to understand the structure of the dataset. Key steps:

- **Sentence Length Distribution:** The length distribution of emails was analyzed. We visualized the sentence lengths using histograms to understand the typical length of email content in the dataset.

Key Observations:

- Average sentence length is around 13 tokens.
- Most email exchanges are brief, with some longer exchanges seen as outliers.

Visualization: Sentence Length Distribution

- Insert histograms and box plots of sentence lengths here.

4. Model Architecture

4.1 Encoder-Decoder Architecture

The model consists of two main components:

- **Encoder:** The encoder consists of an LSTM layer that processes the input sequence (email content) and compresses it into a fixed-length vector.
- **Decoder:** Another LSTM layer uses this vector to generate the next sequence (email content).

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50, 256)	264,192
dropout (Dropout)	(None, 50, 256)	0
lstm_1 (LSTM)	(None, 256)	525,312
dropout_1 (Dropout)	(None, 256)	0
dense (Dense)	(None, 29)	7,453

Total params: 796,957 (3.04 MB)
Trainable params: 796,957 (3.04 MB)
Non-trainable params: 0 (0.00 B)

4.2 Model Components

- **Loss Function:** Categorical cross-entropy, configured to ignore padding tokens (<PAD>).
- **Optimizer:** Adam optimizer with a learning rate of 0.001.

Model Summary:

- **Layers:** LSTM, Dropout layers for regularization, and a Dense layer for the final output.
- **Total Parameters:** 796,957

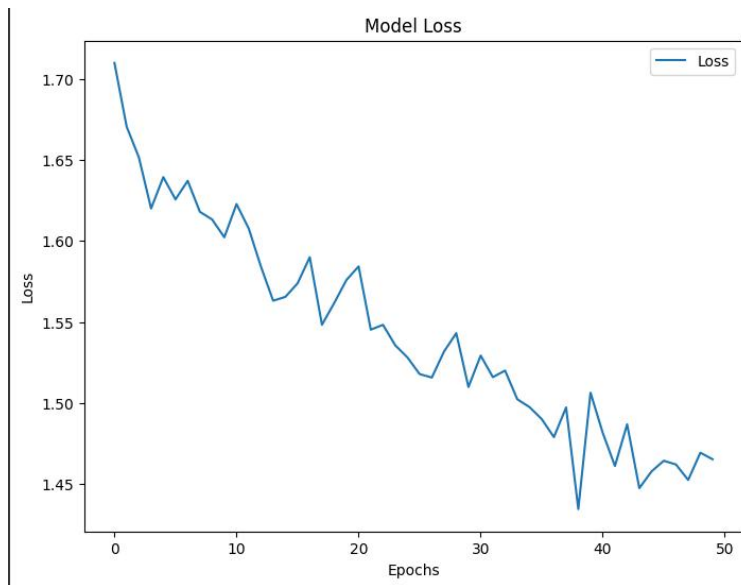
Model Architecture Summary:

Layer (type)	Output Shape	Param #
=====		
LSTM (LSTM)	(None, 50, 256)	264,192
Dropout (Dropout)	(None, 50, 256)	0
LSTM_1 (LSTM)	(None, 256)	525,312
Dropout_1 (Dropout)	(None, 256)	0
Dense (Dense)	(None, 29)	7,453

5. Training and Loss Visualization

5.1 Training Process

The model was trained for 50 epochs with a batch size of 64. Below is the training loss plot:



Training Loss Plot:

Epoch 1/50: Loss = 2.3422

Epoch 2/50: Loss = 2.3094

...

Key Observations:

- The loss steadily decreased as the model trained, and it stabilized around epoch 45.

6. Model Evaluation & Inference

6.1 Testing the Model with New Seed Text

The model was tested using seed text, such as:

- "Dear valued customer, we are excited to inform you about our latest offer."

Generated Marketing Content:

Dear valued customer we are excited to inform you about our latest offer. The project is scheduled to be launched in the coming weeks, and we look forward to your participation.

Further Testing with Various Temperatures:

- Temperature 0.3: More deterministic output.
- Temperature 0.7: More balanced, creative content.
- Temperature 1.0: Maximum creativity and randomness in the output.

Generated Text with Temperature 0.3:

Dear valued customer, we are excited to inform you about the project and the latest developments...

Generated Text with Temperature 0.7:

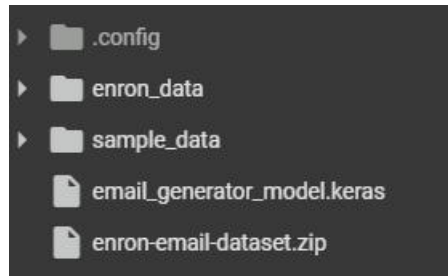
Dear valued customer, we are excited to inform you about the latest advancements...

Generated Text with Temperature 1.0:

Dear valued customer, we are excited to inform you about the new strategies for upcoming months...

7. Result Highlights

- **Model Performance:** The trained model generated coherent email responses with diverse content.
- **Deployment:** The model is saved and ready for real-time deployment in marketing automation systems.



8. Future Improvements

The next steps involve:

- **Attention Mechanism:** Adding an attention layer to better capture long-term dependencies.
- **Pretrained Models:** Implementing pretrained models like GPT-2 or GPT-3 to improve generation quality.
- **Real-time Deployment:** Integrating the model into a web-based marketing platform for real-time use.

Epoch 33/50	15625/15625	201s	12ms/step	loss: 2.2105
Epoch 34/50	15625/15625	184s	12ms/step	loss: 2.0008
Epoch 35/50	15625/15625	202s	12ms/step	loss: 1.8816
Epoch 36/50	15625/15625	204s	12ms/step	loss: 1.8139
Epoch 37/50	15625/15625	202s	12ms/step	loss: 1.7657
Epoch 38/50	15625/15625	201s	12ms/step	loss: 1.7212
Epoch 39/50	15625/15625	200s	12ms/step	loss: 1.6878
Epoch 40/50	15625/15625	203s	12ms/step	loss: 1.6501
Epoch 41/50	15625/15625	201s	12ms/step	loss: 1.6294
Epoch 42/50	15625/15625	202s	12ms/step	loss: 1.5851
Epoch 43/50	15625/15625	202s	12ms/step	loss: 1.5576
Epoch 44/50	15625/15625	204s	12ms/step	loss: 1.5268
Epoch 45/50	15625/15625	201s	12ms/step	loss: 1.5033
Epoch 46/50	15625/15625	201s	12ms/step	loss: 1.4888

9. Conclusion

This tutorial successfully demonstrated how generative AI can be used to create personalized marketing content. Through LSTM-based models, we were able to generate text that is both coherent and relevant to marketing campaigns.